

Advanced Generic Definitions

[BGA]

Example 1 : Generic Three - Way Product

Generic function for three-component tuples:

$[X, Y, Z]$	$\begin{aligned} &triple : X \times Y \times Z \rightarrow X \times Y \times Z \\ &getFirst : X \times Y \times Z \rightarrow X \\ &getSecond : X \times Y \times Z \rightarrow Y \\ &getThird : X \times Y \times Z \rightarrow Z \end{aligned}$
	$\begin{aligned} &\forall x : X; y : Y; z : Z \bullet \\ &\quad triple(x, y, z) = (x, y, z) \wedge \\ &\quad getFirst(x, y, z) = x \wedge \\ &\quad getSecond(x, y, z) = y \wedge \\ &\quad getThird(x, y, z) = z \end{aligned}$

Projection functions for triples.

Example 2 : Generic Binary Tree (Conceptual)

Binary trees are typically defined for specific types in Z notation rather than as fully polymorphic structures. For a concrete type like N, you would define: $Tree ::= leaf \mid node(\mathbb{N} \times Tree \times Tree)$. Generic trees require complex workarounds beyond standard gendef blocks, as free types in fuzz are not easily parameterized.

Example 6 : Generic State Schema

A generic container with capacity: Note: Fuzz supports generic schemas using the `schema[X]` syntax (not `gendef`):

$Container[X]$	$\begin{aligned} &contents : seq\ X \\ &capacity : \mathbb{N} \end{aligned}$
	$\#contents \leq capacity$

This schema is parameterized by the element type X. It can be instantiated as `Container[N]`, `Container[Char]`, etc.

Example 7 : Generic Relation Operations

Relational composition for any types:

$[X, Y, Z]$	$compose : (X \leftrightarrow Y) \times (Y \leftrightarrow Z) \rightarrow (X \leftrightarrow Z)$
	$\begin{aligned} &\forall R : X \leftrightarrow Y; S : Y \leftrightarrow Z \bullet \\ &\quad compose(R, S) = R \circ S \end{aligned}$

Composes two relations using forward composition.

Example 8 : Generic Option Type (Conceptual)

Optional values (like Maybe or Option in other languages) are challenging to express generically in Z notation due to limitations with parameterized free types. For a specific type, you would define: $\text{Option} ::= \text{none} \mid \text{some}\langle T \rangle$. Fully generic option types with operations require advanced patterns beyond standard `gendef` blocks.

Example 9 : Generic Stack Schema

Note: Define generic schema separately, then declare operations in `gendef`:

$\frac{\text{Stack}[X]}{\text{items} : \text{seq } X \quad \text{maxSize} : \mathbb{N}}$
$\# \text{items} \leq \text{maxSize}$
$\frac{[X]}{\text{emptyStack} : \text{Stack}[X] \quad \text{stackSize} : \text{Stack}[X] \rightarrow \mathbb{N}}$
$\text{emptyStack.items} = \langle \rangle \wedge \text{emptyStack.maxSize} = 100$
$\forall s : \text{Stack}[X] \bullet \text{stackSize}(s) = \#s.\text{items}$

Generic stack with query operations. The schema is defined separately using `schema[X]` syntax, then operations use `gendef`.

Example 10 : Generic Pair Schema

Use `schema[X, Y]` syntax for generic schemas:

$\frac{\text{OrderedPair}[X, Y]}{\text{first} : X \quad \text{second} : Y}$

A generic pair type. The schema has two type parameters and can be instantiated as `OrderedPair[N, Char]`, etc.

Example 11 : Generic Zip Function

Combine two sequences into a sequence of pairs:

$\frac{[X, Y]}{\text{zip} : \text{seq } X \times \text{seq } Y \rightarrow \text{seq } (X \times Y)}$
$\text{zip}(\langle \rangle, \langle \rangle) = \langle \rangle$
$\forall x : X; y : Y; xs : \text{seq } X; ys : \text{seq } Y \bullet$
$\text{zip}(\langle x \rangle \hat{\ } xs, \langle y \rangle \hat{\ } ys) =$
$\langle (x, y) \rangle \hat{\ } \text{zip}(xs, ys)$
$\forall xs : \text{seq } X \bullet \text{zip}(xs, \langle \rangle) = \langle \rangle$
$\forall ys : \text{seq } Y \bullet \text{zip}(\langle \rangle, ys) = \langle \rangle$

Pairs up corresponding elements from two sequences.

Example 12 : Practical Example - Generic Database TableGA

Note: Define generic schema separately, then query operations in gendef:

$[Key, Value]$

$TableGA[Key, Value]$ $entries : Key \rightarrow Value$ $size : \mathbb{N}$
$size = \#(\text{dom } entries)$

$[Key, Value]$ $emptyTable : TableGA[Key, Value]$ $tableSize : TableGA[Key, Value] \rightarrow \mathbb{N}$ $allKeys : TableGA[Key, Value] \rightarrow \mathbb{P} Key$
$emptyTable.entries = \{\} \wedge emptyTable.size = 0$ $\forall t : TableGA[Key, Value] \bullet tableSize(t) = t.size$ $\forall t : TableGA[Key, Value] \bullet allKeys(t) = \text{dom } t.entries$

A generic key-value table with query operations. This demonstrates how generic definitions enable reusable, type-safe data structures. The TableGA schema is defined with schema[Key, Value] syntax, allowing proper fuzz typechecking.